

Enhancing Breast Cancer Detection in Mammography Using Vision Transformers (ViT) and CNN Architectures

Matthew Zhou
Department of ECE
New York University
matthewzhou@nyu.edu

Michael Sanfilippo
Department of CS
New York University
mds9427@nyu.edu

Abstract

The detection of breast cancer through mammography is a critical aspect of early diagnosis and improved patient outcomes. This study explores the effectiveness of five state-of-the-art machine learning models—ResNet-50, ResNet-101, EfficientNet, ViT-Base, and ViT-Medium—in analyzing mammographic images. Using the RSNA Breast Cancer Detection dataset from Kaggle [6], the models are evaluated using key performance metrics, specifically Area Under the Curve (AUC) and accuracy. Each model brings its unique advantages: EfficientNet excels in computational efficiency [7], Vision Transformers (ViTs) leverage global attention mechanisms to identify subtle patterns [2], and Convolutional Neural Networks (CNNs) such as ResNet-50 and ResNet-101 effectively capture hierarchical features [4]. By addressing the challenges of false positives and false negatives, this comparative analysis offers insights into the potential clinical applications of these models for automated mammography analysis. Our findings indicate that larger models do not necessarily guarantee better performance, as evidenced by the results of ResNet-50, which outperformed some of its larger counterparts. Our code can be found on <https://github.com/Mattz-CE/rsna> along with best model weights.

Clear Statement of Research Question

Our research aims to determine the effectiveness of Vision Transformers (ViT-Base and ViT-Medium) compared to traditional Convolutional Neural Network (CNN) architectures, specifically ResNet-50, ResNet-101, and EfficientNet, in detecting breast cancer using mammography images. The study evaluates these models based on their ability to achieve high Area Under the Curve (AUC) and accuracy scores when applied to the RSNA Breast Cancer Detection dataset [6]. A key objective is to understand whether Vision Transformer models can outperform traditional CNNs and EfficientNet, particularly in cases involving dense breast tissue where subtle abnormalities are harder to identify. The study further explores how the global attention mechanisms of ViTs compare to the hierarchical feature extraction capabilities of CNNs in improving detection accuracy while

addressing the challenges of false positives and false negatives. Additionally, we examine the computational trade-offs associated with each model to evaluate their suitability for clinical applications, particularly in terms of scalability and efficiency. By answering these questions, this research provides a comprehensive understanding of the strengths and limitations of Vision Transformers, CNNs, and EfficientNet in the context of automated breast cancer detection and highlights their potential for practical implementation in clinical settings.

Hypothesis

This study hypothesizes that Vision Transformer models, specifically ViT-Base and ViT-Medium, will outperform traditional CNN architectures such as ResNet-50, ResNet-101, and the EfficientNet model in detecting breast cancer from mammographic images. Given their global attention mechanisms, ViTs are expected to excel in identifying subtle and complex patterns, particularly in dense breast tissue, leading to superior performance in terms of AUC and accuracy [2]. However, we anticipate that EfficientNet and ResNet models will still provide strong baselines, balancing computational efficiency with feature extraction capabilities [7, 4]. To ensure fair efficiency comparisons among all models, we limited parameter counts below 100 million to create a level playing field.

Sufficient Background for Non-Expert Reader

Breast cancer is one of the most common and serious cancers affecting women worldwide, and early detection plays a key role in improving survival rates [1]. Mammography, which uses low-dose X-rays to examine breast tissue, remains the primary screening tool for detecting breast cancer. By analyzing mammograms, doctors can identify regions of the breast that may appear abnormal, such as lumps or calcifications. However, mammography has limitations. It can sometimes produce false positives, where benign findings are incorrectly identified as cancerous, leading to unnecessary stress and procedures for patients. Conversely, false negatives, where cancerous regions are missed, can delay diagnosis and treatment, reducing the chances of a successful outcome. Machine learning, particularly deep learning, has shown great promise in improving breast cancer de-

tection through mammography. Convolutional Neural Networks (CNNs) are one of the most widely used methods for image analysis tasks. CNNs can analyze images by detecting patterns such as edges, shapes, and textures, and their ability to learn these features hierarchically has made them highly effective for medical imaging [3].

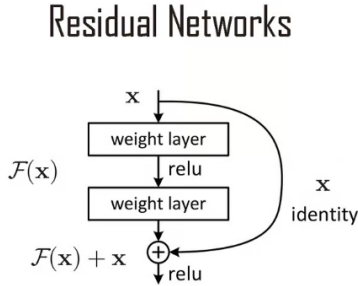


Figure 1: Residual Network Architecture

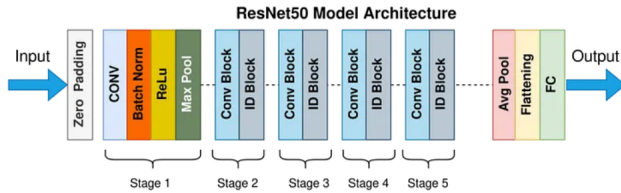


Figure 2: ResNet-50 Architecture

ResNet-50 and ResNet-101, two well-known CNN architectures, were introduced to address the challenge of training very deep networks [4]. Traditional deep networks struggled with the vanishing gradient problem, where the gradient becomes smaller as it propagates through layers, making learning difficult. ResNet solved this issue by introducing residual connections, which allow inputs to skip certain layers and directly influence deeper layers (Figure 1). This innovation enables ResNet-50, with 50 layers, and ResNet-101, with 101 layers, to train deeper networks while efficiently learning complex patterns (Figure 2). ResNet-50 offers a good balance between computational efficiency and performance, while ResNet-101, with its added depth, is better suited for identifying more subtle patterns in complex images [3].

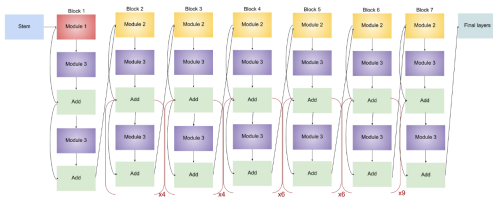


Figure 3: EfficientNet Architecture

EfficientNet builds on the success of CNNs but takes a more streamlined approach to scaling the network. Instead of increasing depth, width, or resolution arbitrarily, EfficientNet uses a compound scaling method that optimizes all three dimensions together [7]. For example, EfficientNet-B6 processes high-resolution images (e.g., 528 x 528 pixels), enabling it to capture finer details that are critical in identifying small or subtle abnormalities in mammograms (Figure 3). Despite its larger input size and higher accuracy, EfficientNet remains computationally efficient compared to other CNN architectures, which makes it a strong candidate for real-world applications in medical imaging [7].

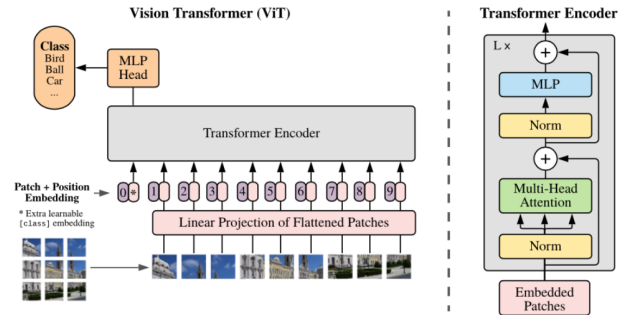


Figure 4: Vision Transformer (ViT) Architecture

Vision Transformers (ViTs) take a different approach by replacing convolutional operations with a self-attention mechanism [8]. ViTs divide an input image into small patches, flatten each patch into a vector, and process these vectors as a sequence. This approach was inspired by transformers, a model architecture initially developed for natural language processing. Unlike CNNs, which are limited to detecting local patterns, Vision Transformers analyze the entire image globally, capturing relationships between distant regions (Figure 4).

ViT-Base and ViT-Medium share the same foundational transformer architecture but differ in scale. ViT-Base has 12 transformer layers, while ViT-Medium increases the number of layers and parameters to improve its capacity to detect complex patterns [2]. This makes ViTs particularly useful for mammography analysis, where understanding global relationships across the image, such as between dense tissue regions and abnormal areas, is essential. However, ViTs typically require large datasets to perform effectively because they lack the inductive biases present in CNNs, which are naturally suited for processing images [2].

The RSNA Breast Cancer Detection dataset from Kaggle is a critical resource for this study [6]. Curated by the Radiological Society of North America, the dataset includes annotated mammography images labeled for cancer detection, along with metadata such as breast density, patient demographics, and mammographic views (e.g., mediolateral oblique and craniocaudal). This dataset provides a real-world benchmark for evaluating the performance of machine learning models in breast cancer detection and helps ensure the clinical relevance of the study [6].

Motivation for Problem Addressed

While mammography is the most widely used screening tool for breast cancer detection, it still faces challenges that limit its accuracy and reliability. Dense breast tissue, in particular, can obscure small or subtle cancerous lesions, making detection difficult even for experienced radiologists. This issue often results in false positives, leading to unnecessary follow-up tests and emotional distress, or false negatives, which delay treatment and reduce survival rates [1].

The variability in radiologist expertise and the fatigue associated with manually analyzing large volumes of mammograms further highlight the need for automated tools that can standardize and improve diagnostic accuracy. Machine learning models, especially deep learning techniques, offer a solution by analyzing mammographic images with high precision and efficiency. However, questions remain about which model architectures are most suitable for this task.

By systematically comparing the performance of Vision Transformers, EfficientNet, and ResNet architectures, this study aims to identify the most effective and reliable approach for automated breast cancer detection. A robust model that balances accuracy, computational efficiency, and clinical applicability has the potential to reduce diagnostic errors, improve early detection rates, and support radiologists by providing consistent, high-quality analysis in real-world settings.

Presentation of Results

Using Parameters Other Than Default

For this study, all hyperparameters were systematically customized to optimize performance for each model. Given the absence of standardized default settings across models, we implemented a unified experimental design that balanced efficiency and performance.

The learning rate was set to $1e-4$ with a cosine decay schedule for smoother convergence, and the batch size was configured to 32 to balance GPU utilization and memory constraints. Early stopping with a patience of 5 epochs and a margin of 0.03 was incorporated to prevent overfitting and improve generalization. These parameters were applied consistently across all models for fairness.

We systematically explored various combinations of hyperparameters, including learning rates, batch sizes, patch sizes (for Vision Transformers), and the number of workers for data loading. Each combination was tested across multiple experiments to identify the optimal settings for each mode.

We tested the interactions between these hyperparameters to assess their impact on model performance. For Vision Transformers, we experimented with patch sizes of 14, 16, and 28 to balance computational complexity and image granularity [2]. For CNN-based models like ResNet and EfficientNet, batch sizes and learning rates were varied extensively to determine the ideal combination for fast convergence and accurate predictions [4, 7].

Additionally, unfreezing layers progressively allowed us to fine-tune the pretrained models effectively, avoiding overfitting while maintaining computational efficiency. The

Table 1: Hyperparameters and Configuration Settings

Parameter	Values
Model Architectures	ResNet-50, ResNet-101, EfficientNet, ViT-Base, ViT-Medium
Batch Size per GPU	8, 16, 32, 48, 64
Patch Size	14, 16, 28
Number of Workers	2, 4, 6, 10, 12, 16
Unfrozen Layers	1, 2, 3, 4, 5
Image Size	512
Maximum Epochs	35
Learning Rate	0.002, 0.001, 0.005, 0.0001
Validation Split	0.2
Early Stopping Margin	0.03
Early Stopping Patience	3

Table 2: Hyperparameters and Configuration Settings for ResNet-50

Parameter	Values
Model Name	ResNet50
Batch Size per GPU	32
Patch Size	16
Number of Workers	16
Unfrozen Layers	3
Image Size	512
Maximum Epochs	35
Learning Rate	0.001
Validation Split	0.2
ES-Margin	0.03
ES-Patience	3

number of workers was adjusted to speed up data loading and minimize GPU idle time.

By systematically combining and evaluating these hyperparameters, we identified the configurations that yielded the best validation AUC and accuracy for each model, ensuring a fair and optimized comparison across architectures. Table 1 summarizes the overall hyperparameter search space, while Table 2 details the final set of parameters that produced the best results.

Display and Interpretation of Model Performances

The analysis of the provided results, as illustrated in Figures 6, 7, 8, 9, and 10, reveals several key insights into the performance of different models, specifically ResNet-50, ResNet-101, EfficientNet, Vision Transformer Base, and Vision Transformer Medium. Each model’s training and validation accuracy, as well as AUC (Area Under the Curve), were scrutinized to understand their learning capabilities and generalization performance.

ResNet-50’s (Figure 6) validation accuracy decreased after initial improvement, stabilizing around 80%, while the training accuracy continued to increase, indicating overfitting. The training AUC improved steadily, reaching 75%, but the validation AUC plateaued early and decreased slightly

to around 68%. This model showed strong learning on the training data but struggled to generalize to the validation data.

ResNet-101 (Figure 7) exhibited dramatic fluctuations in validation accuracy, especially in the early and middle epochs, indicating instability. The training accuracy remained high, around 94% or more, suggesting overfitting. The AUC for this model remained unstable but showed occasional peaks in validation, reaching around 58%. ResNet-101 faced similar challenges as ResNet-50 but with greater instability in validation performance.

EfficientNet (Figure 8) showed a clear drop in validation accuracy after a few epochs, stabilizing between 65% and 80%, while the training accuracy remained stable and higher, indicating overfitting. The training AUC increased consistently, but the validation AUC fluctuated and did not improve significantly, stabilizing around 66%. This suggests that while EfficientNet demonstrated consistent training improvement, it failed to generalize well to the validation data.

The Vision Transformer Base model’s (Figure 9) validation accuracy stabilized early and remained close to the training accuracy, around 98%, with a few sharp dips suggesting intermittent overfitting. The training AUC improved steadily, but the validation AUC plateaued at around 60%, indicating underperformance. This model learned progressively but generalized poorly, achieving a low AUC.

For the Vision Transformer Medium (Figure 10), the validation AUC exhibited significant fluctuations across epochs, indicating instability. The training AUC, although slightly more stable, did not consistently surpass 0.5, suggesting poor learning capability. The validation accuracy plateaued around 98% early in the training process, which is indicative of overfitting, as the training accuracy lagged behind, confirming poor generalization. This model struggled with generalization, likely due to insufficient data or the complexity of the model.

Overall, ResNet-50 achieved the best AUC on validation data among the models tested. All models exhibited varying degrees of overfitting, with training accuracy and AUC improving while validation metrics stagnated or dropped. Vision Transformers, in particular, showed more instability compared to Convolutional Neural Networks (CNNs).

Proper Separation of Training, Validation, and Test Sets

The dataset was partitioned into training (80%) and validation (20%), ensuring stratified sampling to maintain class distribution across all splits. This separation was maintained throughout the experimentation to prevent data leakage and ensure reliable performance metrics. Validation also helped with early stopping for determining the optimal number of epochs to prevent overfitting.

Was the Correct Metric Used?

We primarily used AUC-ROC as our main evaluation metric due to its robustness in handling class imbalance, which is inherent in medical imaging datasets [5]. While F1 scores were also calculated, AUC provides a more comprehensive

view of model performance across different classification thresholds.

Discussion/Conclusion

The results of this study provide important insights into the performance and limitations of the tested models. Table 3 summarizes the key metrics, including validation accuracy, AUC, validation loss, number of epochs, and total parameters for ResNet-50, ResNet-101, EfficientNet, ViT-Base, and ViT-Medium.

Table 3: Best Validation Metrics for Each Model

Model	Val Acc	Val AUC	Epoch	Params
resnet50	0.740632	0.741148	6	23.5M
resnet101	0.914092	0.584653	14	42.5M
efficientnet	0.791354	0.661808	7	20.2M
vit_base	0.977244	0.617185	28	86.0M
vit_medium	0.978800	0.549083	25	63.7M

Among all models, ResNet-50 demonstrated the strongest performance in terms of validation AUC, achieving a score of 0.741. The AUC is particularly significant in this context, as it evaluates the model’s ability to distinguish between cancerous and non-cancerous cases. This makes AUC a far more reliable indicator for imbalanced datasets like the RSNA Breast Cancer Detection dataset, where 97.88% of samples belong to the non-cancerous (negative) class. Despite its relatively smaller size of 23.5 million parameters, ResNet-50’s robust performance highlights the importance of efficient architectural design in handling edge cases and imbalanced class distributions. While ViT-Medium achieved the highest validation accuracy of 97.88%, this result must be interpreted cautiously. Given the significant class imbalance in the dataset, accuracy becomes an unreliable performance metric. A model that simply predicts the majority class (non-cancerous) for all samples could achieve high accuracy without contributing meaningful clinical insights. This explains why Vision Transformers, despite their strong accuracy, failed to surpass ResNet-50 in AUC, which is a more suitable measure for imbalanced classification tasks. Similarly, ViT-Base achieved high accuracy but delivered a lower AUC, further demonstrating that increased model complexity and parameter count do not necessarily translate to better performance in critical evaluation metrics like AUC. In terms of convergence speed and computational efficiency, ResNet-50 and EfficientNet emerged as the most resource-efficient models. ResNet-50 required only 6 epochs, while EfficientNet achieved its best performance in 7 epochs. EfficientNet, with the smallest parameter count of 20.1 million, further demonstrated the advantages of compound scaling in achieving a balance between accuracy, efficiency, and computational demands.

The relationship between model size and AUC, visualized in Figure 5, reinforces these observations. Despite their larger parameter counts and greater complexity, ViT-Base and ViT-Medium failed to outperform ResNet-50, which achieved the best AUC with fewer computational resources. This finding highlights that architectural efficiency, rather

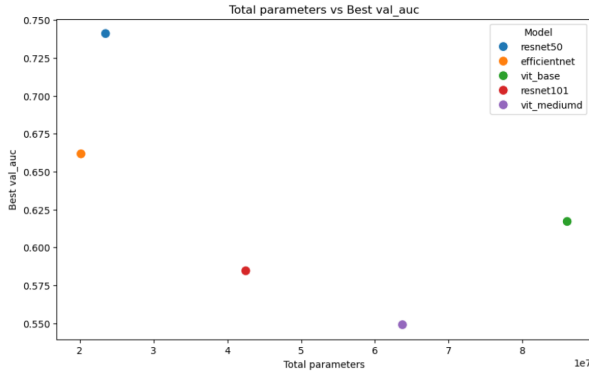


Figure 5: Scatter plot showing the relationship between model size and AUC

than brute-force complexity, can be more impactful for specialized tasks like breast cancer detection. Our initial hypothesis, which proposed that Vision Transformers would outperform CNN-based architectures was disproven. While Vision Transformers excelled in achieving high validation accuracy, this success was largely attributed to the dataset’s class imbalance, where non-cancerous samples dominated. ResNet-50, on the other hand, emerged as the most effective model by achieving the highest AUC, demonstrating its ability to robustly separate cancerous from non-cancerous cases, even in an imbalanced dataset. This study underscores the importance of using appropriate evaluation metrics, such as AUC, for tasks involving imbalanced datasets. While Vision Transformers, particularly ViT-Base and ViT-Medium, showed promise in terms of accuracy, their inability to achieve competitive AUC scores highlights the need for careful performance analysis beyond surface-level metrics. ResNet-50’s superior AUC, combined with its smaller size, faster convergence, and lower computational requirements, makes it the most suitable model for breast cancer detection in this study.

Discussion of Limitations and Alternative Interpretations

Several limitations must be acknowledged to ensure a balanced understanding of the study’s findings.

Firstly, computational constraints limited all models to a parameter count of less than 100 million, potentially hindering the full exploration of Vision Transformers’ (ViTs) potential, as they generally require higher parameter counts for optimal performance compared to Convolutional Neural Networks (CNNs).

Secondly, the RSNA Breast Cancer Detection dataset, despite its clinical relevance, contains inherent imbalances. The low number of positive cases compared to negatives may have influenced model performance, particularly for precision and recall, with outcomes possibly attributed to dataset bias rather than model superiority.

Models like ResNet and EfficientNet are sensitive to input sizes, while Vision Transformers rely on patch embed-

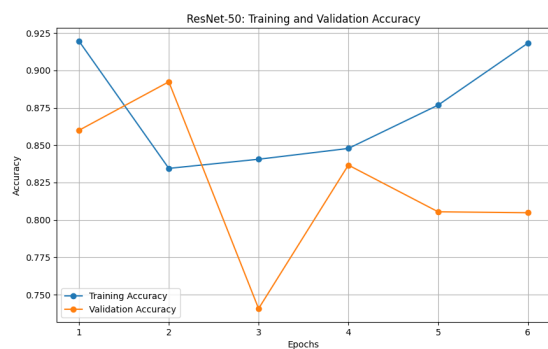
dings, making performance comparisons partially dependent on preprocessing decisions. Additionally, using only single-view mammograms may not capture the full anatomical details available in multi-view imaging, and results could vary significantly with additional perspectives (e.g., cranio-caudal and mediolateral oblique). The exclusion of patient clinical data (e.g., age, breast density, family history) limits the models’ ability to contextualize findings, and including such metadata could improve performance, particularly in ambiguous cases. While pretrained models (ImageNet) were fine-tuned, domain-specific datasets like RSNA may not fully align with general visual datasets, potentially hindering model adaptation to mammographic image patterns.

Clear Statement of Future Work

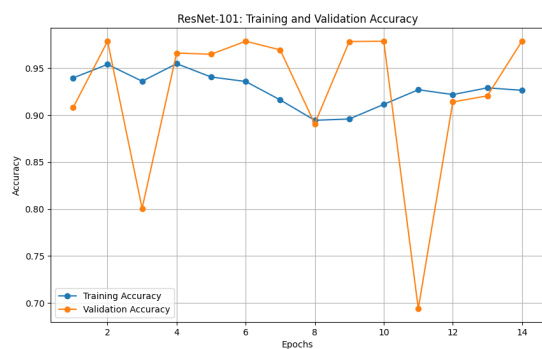
Future research efforts should aim to address the limitations identified and further expand the scope of this study. Investigating hybrid models that combine Vision Transformers’ global attention with CNNs’ hierarchical feature extraction to balance performance and efficiency is a promising direction. Expanding the analysis to incorporate multiple mammographic views (e.g., craniocaudal and mediolateral oblique) could improve detection accuracy and provide a more comprehensive evaluation. Integrating patient demographics, breast density, and family history into the models could enhance prediction capabilities and contextualize results.

Developing lightweight and efficient attention mechanisms for Vision Transformers to reduce computational overhead while retaining performance benefits is another possible area to explore upon. Exploring Vision Transformer architectures with higher base parameter counts and larger pretraining datasets could fully exploit their potential for mammography analysis. Conducting clinical validation of the models on prospective datasets would evaluate their practical applicability in real-world diagnostic workflows. Finally, implementing advanced search strategies (e.g., Bayesian optimization) to identify optimal configurations for learning rates, patch sizes, and layer freezing could further enhance model performance.

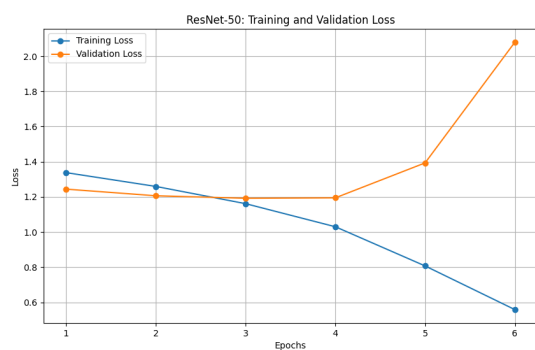
By addressing these areas, future work can further advance the potential of AI-driven mammography analysis for improved breast cancer detection and clinical decision support.



(a) Accuracy



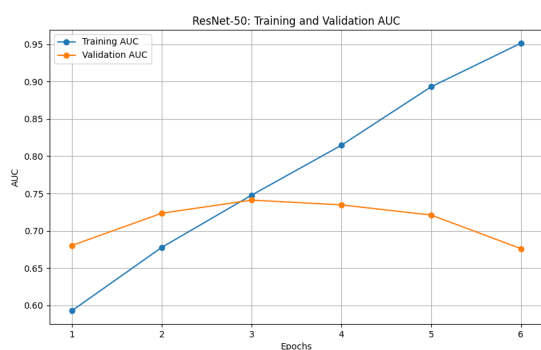
(a) Accuracy



(b) Loss



(b) Loss



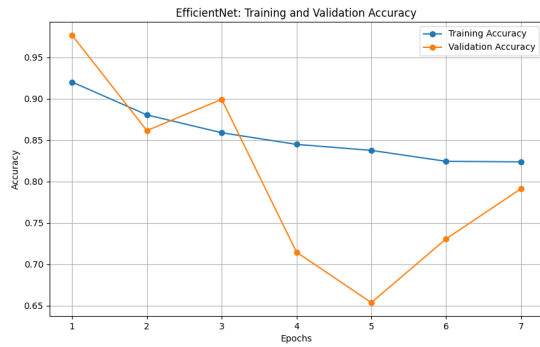
(c) AUC



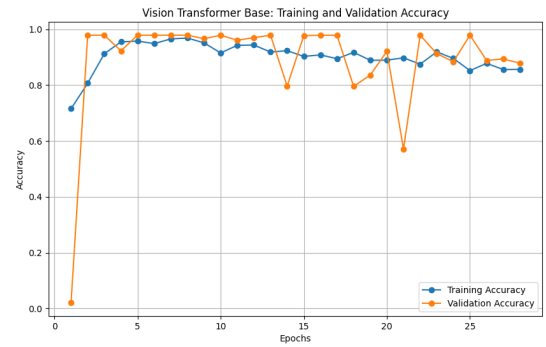
(c) AUC

Figure 6: Performance metrics for ResNet-50 model over 6 epochs

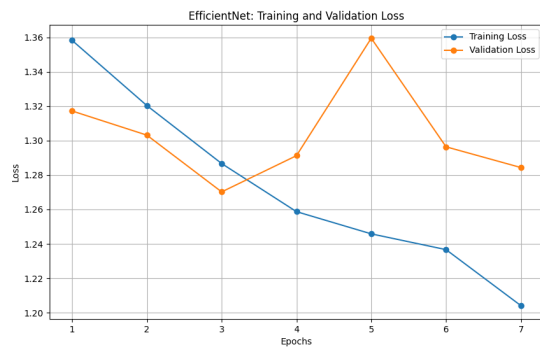
Figure 7: Performance metrics for ResNet-101 model over 14 epochs



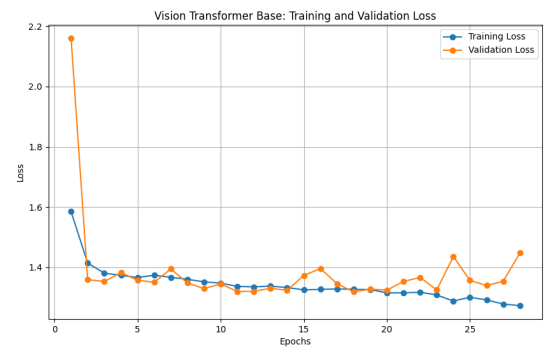
(a) Accuracy



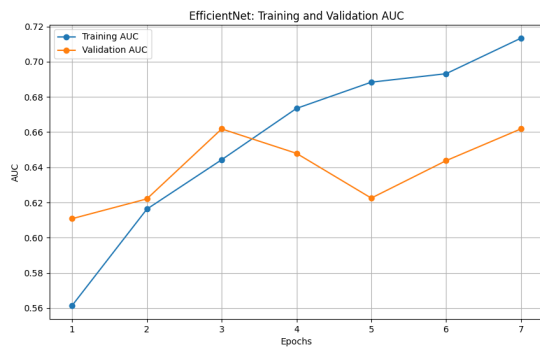
(a) Accuracy



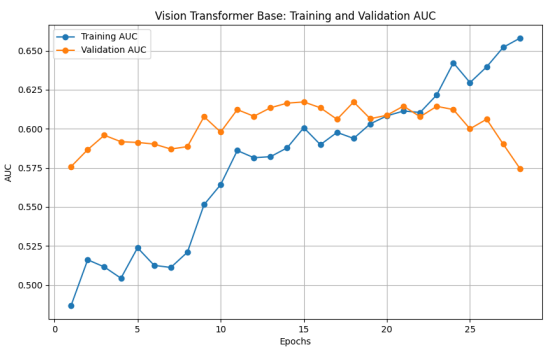
(b) Loss



(b) Loss



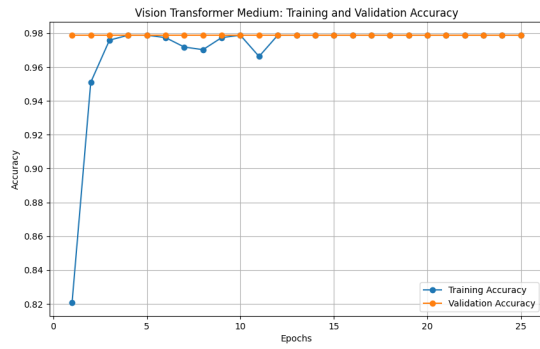
(c) AUC



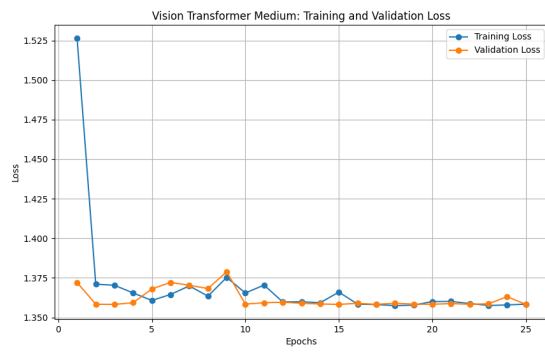
(c) AUC

Figure 8: Performance metrics for EfficientNet model over 7 epochs

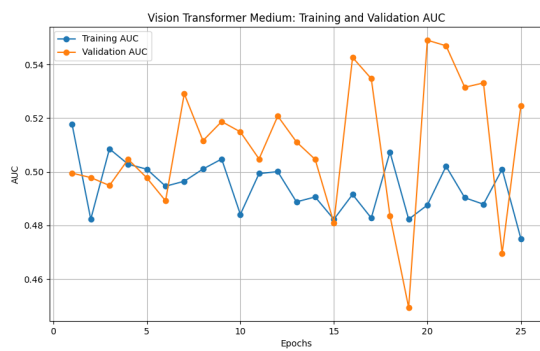
Figure 9: Performance metrics for ViT Base model over 28 epochs



(a) Accuracy



(b) Loss



(c) AUC

Figure 10: Performance metrics for ViT MediumD model over 26 epochs

References

- [1] ACS. Breast cancer facts & figures 2023, 2023.
- [2] Alexey Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. Alexey Dosovitskiy.
- [3] Ian Goodfellow et al. *Deep Learning*. MIT Press, 2016. Ian Goodfellow.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. Corresponding author: Kaiming He.
- [5] Jeffrey M Johnson and Taghi M Khoshgoftaar. A survey on deep learning with class imbalance. *Journal of Big Data*, 2019. Jeffrey M Johnson.
- [6] RSNA. Rsna breast cancer detection dataset. 2022. RSNA.
- [7] Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning (ICML)*, 2019. Mingxing Tan.
- [8] Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017. Ashish Vaswani.

```
!pip install scikit-learn pandas kagglehub torchsummary tensorboard
!wget https://raw.githubusercontent.com/Mattz-CE/rsna/refs/heads/main/payload/train.csv
```

```
Requirement already satisfied: scikit-learn in
/opt/conda/lib/python3.10/site-packages (1.2.2)
Requirement already satisfied: pandas in
/opt/conda/lib/python3.10/site-packages (2.2.3)
Requirement already satisfied: kagglehub in
/opt/conda/lib/python3.10/site-packages (0.3.4)
Collecting torchsummary
  Downloading torchsummary-1.5.1-py3-none-any.whl.metadata (296 bytes)
```

```
Requirement already satisfied: tensorboard in
/opt/conda/lib/python3.10/site-packages (2.16.2)
Requirement already satisfied: numpy>=1.17.3 in
/opt/conda/lib/python3.10/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.3.2 in
/opt/conda/lib/python3.10/site-packages (from scikit-learn) (1.14.1)
Requirement already satisfied: joblib>=1.1.1 in
/opt/conda/lib/python3.10/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/opt/conda/lib/python3.10/site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/opt/conda/lib/python3.10/site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
/opt/conda/lib/python3.10/site-packages (from pandas) (2024.1)
Requirement already satisfied: packaging in
/opt/conda/lib/python3.10/site-packages (from kagglehub) (21.3)
Requirement already satisfied: requests in
/opt/conda/lib/python3.10/site-packages (from kagglehub) (2.32.3)
Requirement already satisfied: tqdm in /opt/conda/lib/python3.10/site-
packages (from kagglehub) (4.66.4)
Requirement already satisfied: absl-py>=0.4 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (1.4.0)
Requirement already satisfied: grpcio>=1.48.2 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (1.62.2)
Requirement already satisfied: markdown>=2.6.8 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (3.6)
Requirement already satisfied: protobuf!=4.24.0,>=3.19.6 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (3.20.3)
Requirement already satisfied: setuptools>=41.0.0 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (70.0.0)
Requirement already satisfied: six>1.9 in
/opt/conda/lib/python3.10/site-packages (from tensorboard) (1.16.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /opt/conda/lib/python3.10/site-packages (from tensorboard) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
```

```
/opt/conda/lib/python3.10/site-packages (from tensorboard) (3.1.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/opt/conda/lib/python3.10/site-packages (from werkzeug>=1.0.1-
>tensorboard) (2.1.5)
Requirement already satisfied: pyparsing!=3.0.5,>=2.0.2 in
/opt/conda/lib/python3.10/site-packages (from packaging->kagglehub)
(3.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/opt/conda/lib/python3.10/site-packages (from requests->kagglehub)
(3.3.2)
Requirement already satisfied: idna<4,>=2.5 in
/opt/conda/lib/python3.10/site-packages (from requests->kagglehub)
(3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.10/site-packages (from requests->kagglehub)
(1.26.18)
Requirement already satisfied: certifi>=2017.4.17 in
/opt/conda/lib/python3.10/site-packages (from requests->kagglehub)
(2024.6.2)
Downloading torchsummary-1.5.1-py3-none-any.whl (2.8 kB)
Installing collected packages: torchsummary
Successfully installed torchsummary-1.5.1
--2024-12-08 23:16:57--
https://raw.githubusercontent.com/Mattz-CE/rsna/refs/heads/main/payload/train.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
185.199.108.133, 185.199.110.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|
185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2559541 (2.4M) [text/plain]
Saving to: 'train.csv'
```

```
train.csv          100%[=====>]    2.44M  ---KB/s    in
0.02s
```

```
2024-12-08 23:16:57 (104 MB/s) - 'train.csv' saved [2559541/2559541]
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models
from torchsummary import summary as ts
from torchinfo import summary
import timm
```

```
##### RESNET #####
```

```
class RESNETBaseline(nn.Module):
    def __init__(self, img_size=None, patch_size=None, **kwargs):
```

```

        super(RESNETBaseline, self).__init__()
        self.resnet =
models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        original_conv = self.resnet.conv1
        self.resnet.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2,
padding=3, bias=False)

        with torch.no_grad():
            self.resnet.conv1.weight.data =
original_conv.weight.data.mean(dim=1, keepdim=True)

        # Modify final fully connected layer
        self.resnet.fc = nn.Linear(self.resnet.fc.in_features, 1)
        self.sigmoid = nn.Sigmoid()

        # Freeze all layers by default
        self._freeze_layers()

def _freeze_layers(self):
    """Freeze all layers except the final classifier"""
    for param in self.resnet.parameters():
        param.requires_grad = False
    # Unfreeze the final fully connected layer
    for param in self.resnet.fc.parameters():
        param.requires_grad = True

def unfreeze_last_n_layers(self, n=3):
    """Unfreeze the last n Bottleneck modules of ResNet"""
    # First freeze everything
    self._freeze_layers()

    # Get the last n Bottleneck modules from layer4
    bottlenecks = list(self.resnet.layer4)[-n:]

    # Unfreeze only these Bottleneck modules
    for bottleneck in bottlenecks:
        for param in bottleneck.parameters():
            param.requires_grad = True

    # Always ensure fc layer is unfrozen
    for param in self.resnet.fc.parameters():
        param.requires_grad = True

def forward(self, x):
    x = self.resnet(x)
    return self.sigmoid(x)

class RESNET101(nn.Module):
    def __init__(self, img_size=None, patch_size=None, **kwargs):
        super(RESNET101, self).__init__()

```

```

        self.resnet =
models.resnet101(weights=models.ResNet101_Weights.IMAGENET1K_V2)
        original_conv = self.resnet.conv1
        self.resnet.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2,
padding=3, bias=False)

        with torch.no_grad():
            self.resnet.conv1.weight.data =
original_conv.weight.data.mean(dim=1, keepdim=True)

        # Modify final fully connected layer
        self.resnet.fc = nn.Linear(self.resnet.fc.in_features, 1)
        self.sigmoid = nn.Sigmoid()

        # Freeze all layers by default
        self._freeze_layers()

def _freeze_layers(self):
    """Freeze all layers except the final classifier"""
    for param in self.resnet.parameters():
        param.requires_grad = False
    # Unfreeze the final fully connected layer
    for param in self.resnet.fc.parameters():
        param.requires_grad = True

def unfreeze_last_n_layers(self, n=3):
    """Unfreeze the last n Bottleneck modules of ResNet"""
    # First freeze everything
    self._freeze_layers()

    # Get the last n Bottleneck modules from layer4
    bottlenecks = list(self.resnet.layer4)[-n:]

    # Unfreeze only these Bottleneck modules
    for bottleneck in bottlenecks:
        for param in bottleneck.parameters():
            param.requires_grad = True

    # Always ensure fc layer is unfrozen
    for param in self.resnet.fc.parameters():
        param.requires_grad = True

def forward(self, x):
    x = self.resnet(x)
    return self.sigmoid(x)

##### EfficientNetV2 #####
class RSNAEfficientNetV2(nn.Module):
    def __init__(self, img_size=None, patch_size=None,
pretrained_weights=None, **kwargs):

```

```

        super(RSNAEfficientNetV2, self).__init__()
        self.model =
models.efficientnet_v2_s(weights=models.EfficientNet_V2_S_Weights.IMAG
ENET1K_V1)

        # Modify first conv layer for single channel input
        original_conv = self.model.features[0][0]
        self.model.features[0][0] = nn.Conv2d(1, 24, kernel_size=3,
stride=2, padding=1, bias=False)

        # Average the weights of the first conv layer
        with torch.no_grad():
            self.model.features[0][0].weight.data =
original_conv.weight.data.mean(dim=1, keepdim=True)

        # Modify classifier for binary output
        num_ftrs = self.model.classifier[-1].in_features
        self.model.classifier[-1] = nn.Linear(num_ftrs, 1)
        self.sigmoid = nn.Sigmoid()

        # Freeze all layers by default
        self._freeze_layers()

    def _freeze_layers(self):
        """Freeze all layers except the classifier"""
        for param in self.model.features.parameters():
            param.requires_grad = False
        # Unfreeze the classifier
        for param in self.model.classifier.parameters():
            param.requires_grad = True

    def unfreeze_last_n_layers(self, n=3):
        """Unfreeze the last n MBConv blocks"""
        # First freeze everything
        self._freeze_layers()

        # Find the last n MBConv blocks
        mbconv_blocks = []
        for module in self.model.modules():
            if isinstance(module, (models.efficientnet.MBConv,
models.efficientnet.FusedMBConv)):
                mbconv_blocks.append(module)

        # Unfreeze only the last n MBConv blocks
        for block in mbconv_blocks[-n:]:
            for param in block.parameters():
                if not isinstance(param, nn.BatchNorm2d):
                    param.requires_grad = True

```

```

    # Always unfreeze classifier
    for param in self.model.classifier.parameters():
        param.requires_grad = True

def forward(self, x):
    x = self.model(x)
    return self.sigmoid(x)

##### RSNAViTBase #####
class RSNAViTBase(nn.Module):
    def __init__(self, img_size=512, patch_size=16, **kwargs):
        super(RSNAViTBase, self).__init__()
        # Ensure img_size is compatible with patch_size
        assert img_size % patch_size == 0, f"Image size {img_size}
must be divisible by patch size {patch_size}"

        # Load pretrained ViT
        self.model = timm.create_model(
            'vit_base_patch16_224',
            pretrained=True,
            num_classes=1,
            img_size=img_size,
            patch_size=patch_size
        )

        # Modify patch embedding for single channel input
        original_patch_embed = self.model.patch_embed.proj
        self.model.patch_embed.proj = nn.Conv2d(
            1, original_patch_embed.out_channels,
            kernel_size=patch_size, stride=patch_size
        )

        # Initialize new patch embedding with average of pretrained
weights
        with torch.no_grad():
            self.model.patch_embed.proj.weight.data =
original_patch_embed.weight.data.mean(
                dim=1, keepdim=True
            )

        # Add sigmoid activation for binary classification
        self.sigmoid = nn.Sigmoid()

        # Freeze all layers by default
        self._freeze_layers()

def _freeze_layers(self):
    """Freeze all layers except the head"""
    for param in self.model.parameters():

```

```

        param.requires_grad = False
    # Unfreeze the classification head
    for param in self.model.head.parameters():
        param.requires_grad = True

def unfreeze_last_n_layers(self, n=3):
    """Unfreeze the last n transformer blocks and the head"""
    # First freeze everything
    self._freeze_layers()

    # Unfreeze the last n transformer blocks
    for block in self.model.blocks[-n:]:
        for param in block.parameters():
            param.requires_grad = True

    # Always unfreeze head
    for param in self.model.head.parameters():
        param.requires_grad = True

    # Unfreeze layer norm layers
    for param in self.model.norm.parameters():
        param.requires_grad = True

def forward(self, x):
    x = self.model(x)
    return self.sigmoid(x)

##### RSNAViTMediumD #####
class RSNAViTMediumD(nn.Module):
    def __init__(self, img_size=512, patch_size=16, **kwargs):
        super(RSNAViTMediumD, self).__init__()
        # Ensure img_size is compatible with patch_size
        assert img_size % patch_size == 0, f"Image size {img_size}
must be divisible by patch size {patch_size}"

        # Load pretrained ViT MediumD with registers and global
average pooling
        self.model = timm.create_model(
'vit_mediumd_patch16_reg4_gap_384.sbb2_e200_in12k_ft_in1k',
    pretrained=True,
    num_classes=1,
    img_size=img_size,
    patch_size=patch_size
)

    # Modify patch embedding for single channel input
    original_patch_embed = self.model.patch_embed.proj
    self.model.patch_embed.proj = nn.Conv2d(

```



```

        1, original_patch_embed.out_channels,
        kernel_size=patch_size, stride=patch_size
    )

    # Initialize new patch embedding with average of pretrained
weights
    with torch.no_grad():
        self.model.patch_embed.proj.weight.data =
original_patch_embed.weight.data.mean(
            dim=1, keepdim=True
        )

    # Add sigmoid activation for binary classification
    self.sigmoid = nn.Sigmoid()

    # Freeze all layers by default
    self._freeze_layers()

def _freeze_layers(self):
    """Freeze all layers except the head"""
    for param in self.model.parameters():
        param.requires_grad = False
    # Unfreeze the classification head
    for param in self.model.head.parameters():
        param.requires_grad = True

def unfreeze_last_n_layers(self, n=3):
    """Unfreeze the last n transformer blocks and the head"""
    # First freeze everything
    self._freeze_layers()

    # Unfreeze the last n transformer blocks
    for block in self.model.blocks[-n:]:
        for param in block.parameters():
            param.requires_grad = True

    # Always unfreeze head
    for param in self.model.head.parameters():
        param.requires_grad = True

    # Unfreeze layer norm layers
    for param in self.model.norm.parameters():
        param.requires_grad = True

def forward(self, x):
    x = self.model(x)
    return self.sigmoid(x)

##### FETCH #####

```

```

def get_model(model_name, **kwargs):
    """Factory function to get the specified model"""
    models = {
        'resnet': RESNETBaseline,
        'resnet101': RESNET101,
        'effinet': RSNAEfficientNetV2,
        'vit_base': RSNAViTBase,
        'vit_mediumd': RSNAViTMediumD
    }

    if model_name not in models:
        raise ValueError(f"Model {model_name} not found. Available models: {list(models.keys())}")

    return models[model_name](**kwargs)

def print_model_summary(model, input_size=(1, 1, 512, 512),
batch_size=1, device="cpu"):
    """Helper function to print model summary with consistent
    formatting"""
    print(f"\n{'='*80}")
    print(f"Model: {model.__class__.__name__}")
    print(f"{'='*80}")

    # Print trainable parameters info
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")
    print(f"Percentage trainable:
{(trainable_params/total_params)*100:.2f}%\n")

    try:
        summary(model, input_size=(batch_size, *input_size[1:]),
device=device, verbose=1)
    except Exception as e:
        print(f>Note: Detailed model summary unavailable for
{model.__class__.__name__}. This is expected for some architectures.")
        print(f"{'='*80}\n")

if __name__ == "__main__":
    # Set device
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    print(f"Using device: {device}")

    # Define standard input size for all models
    INPUT_SIZE = (1, 1, 512, 512) # (batch_size, channels, height,

```

```

width)

# Initialize all models
models_to_test = {
    'resnet': get_model('resnet'),
    'resnet101': get_model('resnet101'),
    'effnet': get_model('effnet'),
    'vit_base': get_model('vit_base'),
    'vit_mediumd': get_model('vit_mediumd')
}

# Print summary for each model
for name, model in models_to_test.items():
    try:
        model = model.to(device)
        print(f"\nTesting {name}:")
        print("-" * 50)

        # Unfreeze last few layers and show summary again
        model.unfreeze_last_n_layers(n=3)
        print("\nModel summary after unfreezing last few layers:")
        print_model_summary(model, input_size=INPUT_SIZE,
device=device)

        # Test forward pass
        dummy_input = torch.randn(INPUT_SIZE).to(device)
        output = model(dummy_input)
        print(f"Output shape: {output.shape}\n")

    except Exception as e:
        print(f"Error testing {name}: {str(e)}\n")
        continue

```

Using device: cuda

```

{"model_id": "c3936ee862a54cd6bcd4c7874136d037", "version_major": 2, "version_minor": 0}

```

Testing ViT-Base:

Model summary after unfreezing last few layers:

```

=====
=====
Model: RSNAViTBase
=====
=====
Total parameters: 86,042,113

```

Trainable parameters: 21,265,921
Percentage trainable: 24.72%

Layer (type:depth-idx)		Output Shape
Param #		
RSNAViTBase		[1, 1]
--		
└─VisionTransformer: 1-1		[1, 1]
787,968		
└┐PatchEmbed: 2-1		[1, 1024, 768]
--		
└┐┐Conv2d: 3-1		[1, 768, 32, 32]
(197,376)		
└┐┐Identity: 3-2		[1, 1024, 768]
--		
└┐┐Dropout: 2-2		[1, 1025, 768]
--		
└┐┐Identity: 2-3		[1, 1025, 768]
--		
└┐┐Identity: 2-4		[1, 1025, 768]
--		
└┐┐Sequential: 2-5		[1, 1025, 768]
--		
└┐┐┐Block: 3-3		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-4		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-5		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-6		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-7		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-8		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-9		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-10		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-11		[1, 1025, 768]
(7,087,872)		
└┐┐┐Block: 3-12		[1, 1025, 768]
7,087,872		
└┐┐┐Block: 3-13		[1, 1025, 768]
7,087,872		

```

|      |      └─Block: 3-14                [1, 1025, 768]
7,087,872
|      └─LayerNorm: 2-6                  [1, 1025, 768]
1,536
|      └─Identity: 2-7                    [1, 768]
--
|      └─Dropout: 2-8                     [1, 768]
--
|      └─Linear: 2-9                      [1, 1]
769
└─Sigmoid: 1-2                          [1, 1]
--

```

```

=====
Total params: 86,042,113
Trainable params: 21,265,921
Non-trainable params: 64,776,192
Total mult-adds (M): 287.17
=====

```

```

=====
Input size (MB): 1.05
Forward/backward pass size (MB): 843.87
Params size (MB): 341.02
Estimated Total Size (MB): 1185.94
=====
=====
=====
=====

```

```

Output shape: torch.Size([1, 1])

```

```

import os
import logging
import json
import shutil
from datetime import datetime
from typing import Dict, Optional
from tqdm import tqdm
from PIL import Image

import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import models, transforms

```

```

from torch.utils.tensorboard import SummaryWriter
from torchsummary import summary

# Checks KAGGLE environment
# KAGGLE = True if os.environ.get('KAGGLE_KERNEL_RUN_TYPE',
# 'Localhost') == 'Interactive' else False
KAGGLE = True if os.path.exists('/kaggle/input') else False

if not KAGGLE:
    from model import get_model

import sys
IN_COLAB = 'google.colab' in sys.modules

# Config
config = {
    'model_name': 'resnet50',
    'batch_size_per_gpu': 32,
    # 16GB GPU → try 16-32
    # 24GB GPU → try 32-48
    # 40GB GPU → try 48-64
    'patch_size': 16, # 16x16 patch size for ViT
    'num_workers' : min(os.cpu_count() * 2 if not IN_COLAB else 4 ,
16) if not KAGGLE else 4,\
    'unfreeze_layers': 3,
    'img_size': 512,
    'epochs': 35,
    'learning_rate': 0.001,
    'validation_split': 0.2,
    'early_stopping_margin': 0.03, # Stop if train_auc - val_auc >
margin
    'early_stopping_patience': 3 # Number of epochs to wait before
stopping
}

# Create initial run directory
run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
run_dir = f"runs/run_{config['model_name']}_{run_id}" # Initial run
directory without epoch count
os.makedirs(run_dir, exist_ok=True)

# Create subdirectories
log_dir = os.path.join(run_dir, "logs")
model_dir = os.path.join(run_dir, "models")
tensorboard_dir = os.path.join(run_dir, "tensorboard")
os.makedirs(log_dir, exist_ok=True)
os.makedirs(model_dir, exist_ok=True)
os.makedirs(tensorboard_dir, exist_ok=True)

# Save config

```

```

with open(os.path.join(run_dir, 'config.json'), 'w') as f:
    json.dump(config, f, indent=4)

# Set up logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler(os.path.join(log_dir, 'training.log')),
        logging.StreamHandler()
    ]
)

# Check CUDA availability
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
num_gpus = torch.cuda.device_count()
logging.info(f"Using device: {device}")
logging.info(f"Number of GPUs: {num_gpus}")

# Update batch size based on number of GPUs
config['total_batch_size'] = config['batch_size_per_gpu'] * max(1,
num_gpus)
logging.info(f"Total batch size: {config['total_batch_size']}")

class RSNADataset(Dataset):
    def __init__(self, df, transform=None, is_training=True):
        self.df = df
        self.transform = transform
        self.is_training = is_training

        # Calculate class weights here
        self.pos_weight = len(df[df['cancer'] == 0]) /
len(df[df['cancer'] == 1])

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['img_path']
        label = self.df.iloc[idx]['cancer']

        image = Image.open(img_path).convert('L') # Only loads gray
scale to 1 channel

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor(label, dtype=torch.float32)

    def get_pos_weight(self):

```

```

        return self.pos_weight

def train_epoch(model, train_loader, criterion, optimizer, device,
epoch):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    all_outputs = []
    all_targets = []

    pbar = tqdm(train_loader, desc=f'Epoch
{epoch+1}/{config["epochs"]} [Train]')

    for inputs, targets in pbar:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs).squeeze()
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        predictions = (outputs >= 0.5).float()
        correct += (predictions == targets).sum().item()
        total += targets.size(0)

        all_outputs.extend(outputs.detach().cpu().numpy())
        all_targets.extend(targets.cpu().numpy())

    try:
        current_auc = roc_auc_score(all_targets, all_outputs)
    except ValueError:
        current_auc = 0.0

    avg_loss = running_loss / (pbar.n + 1)
    current_acc = correct / total
    pbar.set_postfix({
        'loss': f'{avg_loss:.4f}',
        'acc': f'{current_acc:.4f}',
        'auc': f'{current_auc:.4f}'
    })

    epoch_loss = running_loss / len(train_loader)
    epoch_acc = correct / total
    train_auc = roc_auc_score(all_targets, all_outputs)

    return epoch_loss, epoch_acc, train_auc

```



```

def validate(model, val_loader, criterion, device):
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    all_outputs = []
    all_targets = []

    # Initialize progress bar
    pbar = tqdm(val_loader, desc='Validation')

    with torch.no_grad():
        for inputs, targets in pbar:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs).squeeze()
            loss = criterion(outputs, targets)

            val_loss += loss.item()
            predictions = (outputs >= 0.5).float()
            correct += (predictions == targets).sum().item()
            total += targets.size(0)

            all_outputs.extend(outputs.cpu().numpy())
            all_targets.extend(targets.cpu().numpy())

            # Calculate current AUC
            try:
                current_auc = roc_auc_score(all_targets, all_outputs)
            except ValueError: # In case only one class is present in
batch
                current_auc = 0.0

            # Update progress bar with all metrics
            avg_loss = val_loss / (pbar.n + 1)
            current_acc = correct / total
            pbar.set_postfix({
                'loss': f'{avg_loss:.4f}',
                'acc': f'{current_acc:.4f}',
                'auc': f'{current_auc:.4f}'
            })

    val_loss = val_loss / len(val_loader)
    val_acc = correct / total
    val_auc = roc_auc_score(all_targets, all_outputs)

    return val_loss, val_acc, val_auc

def update_run_dir_name(current_run_dir, epoch):

```

```

        """Update run directory name with current epoch count"""
        base_path = os.path.dirname(current_run_dir)
        run_id = os.path.basename(current_run_dir).split('_ep')[0] # Get
original run ID without epoch
        new_run_dir = os.path.join(base_path, f"{run_id}_ep{epoch}")

        if os.path.exists(current_run_dir) and current_run_dir !=
new_run_dir:
            # Create new directories before moving
            new_tensorboard_dir = os.path.join(new_run_dir, "tensorboard")
            new_log_dir = os.path.join(new_run_dir, "logs")
            new_model_dir = os.path.join(new_run_dir, "models")

            os.makedirs(new_run_dir, exist_ok=True)
            os.makedirs(new_tensorboard_dir, exist_ok=True)
            os.makedirs(new_log_dir, exist_ok=True)
            os.makedirs(new_model_dir, exist_ok=True)

            # Move contents from old directories to new ones
            old_tensorboard_dir = os.path.join(current_run_dir,
"tensorboard")
            old_log_dir = os.path.join(current_run_dir, "logs")
            old_model_dir = os.path.join(current_run_dir, "models")

            if os.path.exists(old_tensorboard_dir):
                for item in os.listdir(old_tensorboard_dir):
                    shutil.move(os.path.join(old_tensorboard_dir, item),
new_tensorboard_dir)
            if os.path.exists(old_log_dir):
                for item in os.listdir(old_log_dir):
                    shutil.move(os.path.join(old_log_dir, item),
new_log_dir)
            if os.path.exists(old_model_dir):
                for item in os.listdir(old_model_dir):
                    shutil.move(os.path.join(old_model_dir, item),
new_model_dir)

            # Move any remaining files
            for item in os.listdir(current_run_dir):
                if item not in ["tensorboard", "logs", "models"]:
                    shutil.move(os.path.join(current_run_dir, item),
new_run_dir)

            # Remove old directory
            shutil.rmtree(current_run_dir)

        return new_run_dir

def main():

```

```

global run_dir
if not KAGGLE and not IN_COLAB:
    # Save script to run directory
    shutil.copy(__file__, os.path.join(run_dir, 'script.py'))
    shutil.copy('model.py', os.path.join(run_dir, 'model.py'))

# Data transformations
train_transform = transforms.Compose([
    transforms.Resize((config['img_size'], config['img_size'])),
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5]) # Single channel
normalization
])

val_transform = transforms.Compose([
    transforms.Resize((config['img_size'], config['img_size'])),
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5]) # Single channel
normalization
])

# Load and prepare data
logging.info("Loading and preparing data...")
if KAGGLE:
    base_path = '/kaggle/input/rsna-'+str(config['img_size'])
+ '/train_images_processed_'+str(config['img_size'])
else:
    try:
        import kagglehub
        path = kagglehub.dataset_download("mattzhou/rsna-" +
str(config['img_size']))
        base_path = os.path.join(path,
'train_images_processed_'+str(config['img_size'])+'')
    except:
        base_path =
'/workspace/.cache/kagglehub/datasets/mattzhou/rsna-'+str(config['img_
size'])+ '/versions/1/train_images_processed_'+ str(config['img_size'])
+ '/'

df = pd.read_csv('train.csv')
df['img_path'] = df.apply(lambda row: os.path.join(base_path,
str(row.patient_id), f"{str(row.image_id)}.png"), axis=1)

# Log class distribution
pos_samples = len(df[df['cancer'] == 1])
neg_samples = len(df[df['cancer'] == 0])
logging.info(f"Class distribution - Positive: {pos_samples},
Negative: {neg_samples}")
logging.info(f"Positive to Negative ratio: 1:
{neg_samples/pos_samples:.2f}")

```

```

train_df, val_df = train_test_split(df,
test_size=config['validation_split'], stratify=df['cancer'])

train_dataset = RSNADataSet(train_df, transform=train_transform,
is_training=True)
val_dataset = RSNADataSet(val_df, transform=val_transform,
is_training=False)

train_loader = DataLoader(train_dataset,
batch_size=config['total_batch_size'], shuffle=True,
num_workers=config['num_workers'], pin_memory=True)
val_loader = DataLoader(val_dataset,
batch_size=config['total_batch_size'], shuffle=False,
num_workers=config['num_workers'], pin_memory=True)

# Initialize model
model = get_model(
    config['model_name'],
    img_size=config['img_size'],
    patch_size=config['patch_size'] if 'vit' in
config['model_name'] else None
)
model.unfreeze_last_n_layers(n=config['unfreeze_layers'])

if num_gpus > 1:
    model = nn.DataParallel(model)
    model = model.to(device)

logging.info("Model summary:\n")
logging.info(summary(model, input_size=(1, config['img_size'],
config['img_size'])))

# Use weighted BCE Loss
pos_weight =
torch.tensor([train_dataset.get_pos_weight()]).to(device)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# Remove sigmoid from model since BCEWithLogitsLoss includes it
if isinstance(model, nn.DataParallel):
    model.module.sigmoid = nn.Identity()
else:
    model.sigmoid = nn.Identity()

optimizer = torch.optim.Adam(model.parameters(),
lr=config['learning_rate'])
scheduler = torch.optim.lr_scheduler.ReduceLRonPlateau(
    optimizer, mode='min', patience=3, factor=0.5, verbose=True
)

```

```

# TensorBoard writer
writer = None # Initialize writer as None

# Training loop
best_val_loss = float('inf')
best_val_auc = 0.0
metrics_list = []

# Early stopping variables
early_stop_counter = 0
best_auc_diff = float('inf')

for epoch in range(config['epochs']):
    # Update run directory name with current epoch
    run_dir = update_run_dir_name(run_dir, epoch + 1)

    # Update tensorboard writer for the new directory
    if writer is not None:
        writer.close()
    tensorboard_dir = os.path.join(run_dir, "tensorboard")
    writer = SummaryWriter(tensorboard_dir)

    # Training phase
    train_loss, train_acc, train_auc = train_epoch(
        model, train_loader, criterion, optimizer, device, epoch
    )

    # Validation phase
    val_loss, val_acc, val_auc = validate(model, val_loader,
criterion, device)

    # Calculate AUC difference for early stopping
    auc_diff = train_auc - val_auc

    # Log metrics
    metrics = {
        'epoch': epoch + 1,
        'loss': train_loss,
        'accuracy': train_acc,
        'auc': train_auc,
        'val_loss': val_loss,
        'val_accuracy': val_acc,
        'val_auc': val_auc,
        'auc_diff': auc_diff
    }
    metrics_list.append(metrics)

    # Save metrics with updated path
    pd.DataFrame(metrics_list).to_csv(os.path.join(run_dir,

```

```

'metrics.csv'), index=False)

# TensorBoard logging
writer.add_scalar('Loss/train', train_loss, epoch)
writer.add_scalar('Loss/val', val_loss, epoch)
writer.add_scalar('Accuracy/train', train_acc, epoch)
writer.add_scalar('Accuracy/val', val_acc, epoch)
writer.add_scalar('AUC/train', train_auc, epoch)
writer.add_scalar('AUC/val', val_auc, epoch)
writer.add_scalar('AUC/difference', auc_diff, epoch)

# Save best models based on both validation loss and AUC
if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save(model.state_dict(), os.path.join(run_dir,
'models', 'best_loss_model.pth'))
    logging.info(f"New best validation loss:
{best_val_loss:.4f}")

    if val_auc > best_val_auc:
        best_val_auc = val_auc
        torch.save(model.state_dict(), os.path.join(run_dir,
'models', 'best_auc_model.pth'))
        logging.info(f"New best validation AUC:
{best_val_auc:.4f}")

# Early stopping check
if auc_diff > config['early_stopping_margin']:
    early_stop_counter += 1
    logging.info(f"Early stopping counter:
{early_stop_counter}/{config['early_stopping_patience']}")
    if early_stop_counter >=
config['early_stopping_patience']:
        logging.info(f"Early stopping triggered! Train AUC
exceeds Val AUC by {auc_diff:.4f}")
        break
else:
    early_stop_counter = 0

# Update learning rate based on validation AUC instead of loss
scheduler.step(-val_auc) # Negative because scheduler is in
'min' mode

# Log epoch summary
logging.info(
    f"Epoch {epoch + 1}: "
    f"loss: {train_loss:.4f}, accuracy: {train_acc:.4f}, auc:
{train_auc:.4f}, "
    f"val_loss: {val_loss:.4f}, val_accuracy: {val_acc:.4f},

```

```

val_auc: {val_auc:.4f}, "
        f"auc_diff: {auc_diff:.4f}"
    )

    # Close the final writer
    if writer is not None:
        writer.close()

    # Save final model with updated path
    torch.save(model.state_dict(), os.path.join(run_dir, 'models',
'final_model.pth'))
    logging.info(f"Training completed! All files saved in: {run_dir}")

if __name__ == '__main__':
    main()

```

Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
 100%|██████████| 97.8M/97.8M [00:00<00:00, 232MB/s]
 /opt/conda/lib/python3.10/site-packages/torch/optim/lr_scheduler.py:60
 : UserWarning: The verbose parameter is deprecated. Please use
 get_last_lr() to access the learning rate.
 warnings.warn(

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 256, 256]	3,136
BatchNorm2d-2	[-1, 64, 256, 256]	128
ReLU-3	[-1, 64, 256, 256]	0
MaxPool2d-4	[-1, 64, 128, 128]	0
Conv2d-5	[-1, 64, 128, 128]	4,096
BatchNorm2d-6	[-1, 64, 128, 128]	128
ReLU-7	[-1, 64, 128, 128]	0
Conv2d-8	[-1, 64, 128, 128]	36,864
BatchNorm2d-9	[-1, 64, 128, 128]	128
ReLU-10	[-1, 64, 128, 128]	0
Conv2d-11	[-1, 256, 128, 128]	16,384
BatchNorm2d-12	[-1, 256, 128, 128]	512
Conv2d-13	[-1, 256, 128, 128]	16,384
BatchNorm2d-14	[-1, 256, 128, 128]	512
ReLU-15	[-1, 256, 128, 128]	0
Bottleneck-16	[-1, 256, 128, 128]	0
Conv2d-17	[-1, 64, 128, 128]	16,384
BatchNorm2d-18	[-1, 64, 128, 128]	128
ReLU-19	[-1, 64, 128, 128]	0
Conv2d-20	[-1, 64, 128, 128]	36,864
BatchNorm2d-21	[-1, 64, 128, 128]	128
ReLU-22	[-1, 64, 128, 128]	0

Conv2d-23	[-1, 256, 128, 128]	16,384
BatchNorm2d-24	[-1, 256, 128, 128]	512
ReLU-25	[-1, 256, 128, 128]	0
Bottleneck-26	[-1, 256, 128, 128]	0
Conv2d-27	[-1, 64, 128, 128]	16,384
BatchNorm2d-28	[-1, 64, 128, 128]	128
ReLU-29	[-1, 64, 128, 128]	0
Conv2d-30	[-1, 64, 128, 128]	36,864
BatchNorm2d-31	[-1, 64, 128, 128]	128
ReLU-32	[-1, 64, 128, 128]	0
Conv2d-33	[-1, 256, 128, 128]	16,384
BatchNorm2d-34	[-1, 256, 128, 128]	512
ReLU-35	[-1, 256, 128, 128]	0
Bottleneck-36	[-1, 256, 128, 128]	0
Conv2d-37	[-1, 128, 128, 128]	32,768
BatchNorm2d-38	[-1, 128, 128, 128]	256
ReLU-39	[-1, 128, 128, 128]	0
Conv2d-40	[-1, 128, 64, 64]	147,456
BatchNorm2d-41	[-1, 128, 64, 64]	256
ReLU-42	[-1, 128, 64, 64]	0
Conv2d-43	[-1, 512, 64, 64]	65,536
BatchNorm2d-44	[-1, 512, 64, 64]	1,024
Conv2d-45	[-1, 512, 64, 64]	131,072
BatchNorm2d-46	[-1, 512, 64, 64]	1,024
ReLU-47	[-1, 512, 64, 64]	0
Bottleneck-48	[-1, 512, 64, 64]	0
Conv2d-49	[-1, 128, 64, 64]	65,536
BatchNorm2d-50	[-1, 128, 64, 64]	256
ReLU-51	[-1, 128, 64, 64]	0
Conv2d-52	[-1, 128, 64, 64]	147,456
BatchNorm2d-53	[-1, 128, 64, 64]	256
ReLU-54	[-1, 128, 64, 64]	0
Conv2d-55	[-1, 512, 64, 64]	65,536
BatchNorm2d-56	[-1, 512, 64, 64]	1,024
ReLU-57	[-1, 512, 64, 64]	0
Bottleneck-58	[-1, 512, 64, 64]	0
Conv2d-59	[-1, 128, 64, 64]	65,536
BatchNorm2d-60	[-1, 128, 64, 64]	256
ReLU-61	[-1, 128, 64, 64]	0
Conv2d-62	[-1, 128, 64, 64]	147,456
BatchNorm2d-63	[-1, 128, 64, 64]	256
ReLU-64	[-1, 128, 64, 64]	0
Conv2d-65	[-1, 512, 64, 64]	65,536
BatchNorm2d-66	[-1, 512, 64, 64]	1,024
ReLU-67	[-1, 512, 64, 64]	0
Bottleneck-68	[-1, 512, 64, 64]	0
Conv2d-69	[-1, 128, 64, 64]	65,536
BatchNorm2d-70	[-1, 128, 64, 64]	256
ReLU-71	[-1, 128, 64, 64]	0

Conv2d-72	[-1, 128, 64, 64]	147,456
BatchNorm2d-73	[-1, 128, 64, 64]	256
ReLU-74	[-1, 128, 64, 64]	0
Conv2d-75	[-1, 512, 64, 64]	65,536
BatchNorm2d-76	[-1, 512, 64, 64]	1,024
ReLU-77	[-1, 512, 64, 64]	0
Bottleneck-78	[-1, 512, 64, 64]	0
Conv2d-79	[-1, 256, 64, 64]	131,072
BatchNorm2d-80	[-1, 256, 64, 64]	512
ReLU-81	[-1, 256, 64, 64]	0
Conv2d-82	[-1, 256, 32, 32]	589,824
BatchNorm2d-83	[-1, 256, 32, 32]	512
ReLU-84	[-1, 256, 32, 32]	0
Conv2d-85	[-1, 1024, 32, 32]	262,144
BatchNorm2d-86	[-1, 1024, 32, 32]	2,048
Conv2d-87	[-1, 1024, 32, 32]	524,288
BatchNorm2d-88	[-1, 1024, 32, 32]	2,048
ReLU-89	[-1, 1024, 32, 32]	0
Bottleneck-90	[-1, 1024, 32, 32]	0
Conv2d-91	[-1, 256, 32, 32]	262,144
BatchNorm2d-92	[-1, 256, 32, 32]	512
ReLU-93	[-1, 256, 32, 32]	0
Conv2d-94	[-1, 256, 32, 32]	589,824
BatchNorm2d-95	[-1, 256, 32, 32]	512
ReLU-96	[-1, 256, 32, 32]	0
Conv2d-97	[-1, 1024, 32, 32]	262,144
BatchNorm2d-98	[-1, 1024, 32, 32]	2,048
ReLU-99	[-1, 1024, 32, 32]	0
Bottleneck-100	[-1, 1024, 32, 32]	0
Conv2d-101	[-1, 256, 32, 32]	262,144
BatchNorm2d-102	[-1, 256, 32, 32]	512
ReLU-103	[-1, 256, 32, 32]	0
Conv2d-104	[-1, 256, 32, 32]	589,824
BatchNorm2d-105	[-1, 256, 32, 32]	512
ReLU-106	[-1, 256, 32, 32]	0
Conv2d-107	[-1, 1024, 32, 32]	262,144
BatchNorm2d-108	[-1, 1024, 32, 32]	2,048
ReLU-109	[-1, 1024, 32, 32]	0
Bottleneck-110	[-1, 1024, 32, 32]	0
Conv2d-111	[-1, 256, 32, 32]	262,144
BatchNorm2d-112	[-1, 256, 32, 32]	512
ReLU-113	[-1, 256, 32, 32]	0
Conv2d-114	[-1, 256, 32, 32]	589,824
BatchNorm2d-115	[-1, 256, 32, 32]	512
ReLU-116	[-1, 256, 32, 32]	0
Conv2d-117	[-1, 1024, 32, 32]	262,144
BatchNorm2d-118	[-1, 1024, 32, 32]	2,048
ReLU-119	[-1, 1024, 32, 32]	0
Bottleneck-120	[-1, 1024, 32, 32]	0

Conv2d-121	[-1, 256, 32, 32]	262,144
BatchNorm2d-122	[-1, 256, 32, 32]	512
ReLU-123	[-1, 256, 32, 32]	0
Conv2d-124	[-1, 256, 32, 32]	589,824
BatchNorm2d-125	[-1, 256, 32, 32]	512
ReLU-126	[-1, 256, 32, 32]	0
Conv2d-127	[-1, 1024, 32, 32]	262,144
BatchNorm2d-128	[-1, 1024, 32, 32]	2,048
ReLU-129	[-1, 1024, 32, 32]	0
Bottleneck-130	[-1, 1024, 32, 32]	0
Conv2d-131	[-1, 256, 32, 32]	262,144
BatchNorm2d-132	[-1, 256, 32, 32]	512
ReLU-133	[-1, 256, 32, 32]	0
Conv2d-134	[-1, 256, 32, 32]	589,824
BatchNorm2d-135	[-1, 256, 32, 32]	512
ReLU-136	[-1, 256, 32, 32]	0
Conv2d-137	[-1, 1024, 32, 32]	262,144
BatchNorm2d-138	[-1, 1024, 32, 32]	2,048
ReLU-139	[-1, 1024, 32, 32]	0
Bottleneck-140	[-1, 1024, 32, 32]	0
Conv2d-141	[-1, 512, 32, 32]	524,288
BatchNorm2d-142	[-1, 512, 32, 32]	1,024
ReLU-143	[-1, 512, 32, 32]	0
Conv2d-144	[-1, 512, 16, 16]	2,359,296
BatchNorm2d-145	[-1, 512, 16, 16]	1,024
ReLU-146	[-1, 512, 16, 16]	0
Conv2d-147	[-1, 2048, 16, 16]	1,048,576
BatchNorm2d-148	[-1, 2048, 16, 16]	4,096
Conv2d-149	[-1, 2048, 16, 16]	2,097,152
BatchNorm2d-150	[-1, 2048, 16, 16]	4,096
ReLU-151	[-1, 2048, 16, 16]	0
Bottleneck-152	[-1, 2048, 16, 16]	0
Conv2d-153	[-1, 512, 16, 16]	1,048,576
BatchNorm2d-154	[-1, 512, 16, 16]	1,024
ReLU-155	[-1, 512, 16, 16]	0
Conv2d-156	[-1, 512, 16, 16]	2,359,296
BatchNorm2d-157	[-1, 512, 16, 16]	1,024
ReLU-158	[-1, 512, 16, 16]	0
Conv2d-159	[-1, 2048, 16, 16]	1,048,576
BatchNorm2d-160	[-1, 2048, 16, 16]	4,096
ReLU-161	[-1, 2048, 16, 16]	0
Bottleneck-162	[-1, 2048, 16, 16]	0
Conv2d-163	[-1, 512, 16, 16]	1,048,576
BatchNorm2d-164	[-1, 512, 16, 16]	1,024
ReLU-165	[-1, 512, 16, 16]	0
Conv2d-166	[-1, 512, 16, 16]	2,359,296
BatchNorm2d-167	[-1, 512, 16, 16]	1,024
ReLU-168	[-1, 512, 16, 16]	0
Conv2d-169	[-1, 2048, 16, 16]	1,048,576

BatchNorm2d-170	[-1, 2048, 16, 16]	4,096
ReLU-171	[-1, 2048, 16, 16]	0
Bottleneck-172	[-1, 2048, 16, 16]	0
AdaptiveAvgPool2d-173	[-1, 2048, 1, 1]	0
Linear-174	[-1, 1]	2,049
ResNet-175	[-1, 1]	0
Sigmoid-176	[-1, 1]	0

```
=====
Total params: 23,503,809
Trainable params: 14,966,785
Non-trainable params: 8,537,024
```

```
-----
Input size (MB): 1.00
Forward/backward pass size (MB): 1497.02
Params size (MB): 89.66
Estimated Total Size (MB): 1587.68
-----
```

```
Epoch 1/35 [Train]: 0%|          | 0/1368 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
```

```
    self.pid = os.fork()
Epoch 1/35 [Train]: 100%|██████████| 1368/1368 [07:15<00:00,
3.38it/s, loss=1.3379, acc=0.9196,
auc=0.5931]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
```

```
    self.pid = os.fork()
Epoch 1/35 [Train]: 100%|██████████| 1368/1368 [07:15<00:00,
3.14it/s, loss=1.3379, acc=0.9196, auc=0.5931]
Validation: 0%|          | 0/342 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
```

```
    self.pid = os.fork()
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.39it/s,
loss=1.2440, acc=0.8600,
auc=0.6805]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
```

```
    self.pid = os.fork()
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.30it/s,
loss=1.2440, acc=0.8600, auc=0.6805]
```

```
Epoch 2/35 [Train]: 0%|          | 0/1368 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
```

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 2/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00, 3.38it/s, loss=1.2594, acc=0.8345, auc=0.6779]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 2/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00, 3.16it/s, loss=1.2594, acc=0.8345, auc=0.6779]
Validation: 0%|██████████| 0/342 [00:00<?, ?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.38it/s, loss=1.2062, acc=0.8924, auc=0.7236]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.31it/s, loss=1.2062, acc=0.8924, auc=0.7236]
Epoch 3/35 [Train]: 0%|██████████| 0/1368 [00:00<?, ?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 3/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00, 3.38it/s, loss=1.1613, acc=0.8406, auc=0.7478]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 3/35 [Train]: 100%|██████████| 1368/1368 [07:13<00:00, 3.16it/s, loss=1.1613, acc=0.8406, auc=0.7478]
Validation: 0%|██████████| 0/342 [00:00<?, ?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66: RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

```
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.37it/s,
loss=1.1920, acc=0.7406,
auc=0.7411]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.31it/s,
loss=1.1920, acc=0.7406, auc=0.7411]
Epoch 4/35 [Train]: 0%|██████████| 0/1368 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Epoch 4/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00,
3.37it/s, loss=1.0300, acc=0.8478,
auc=0.8146]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Epoch 4/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00,
3.16it/s, loss=1.0300, acc=0.8478, auc=0.8146]
Validation: 0%|██████████| 0/342 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.38it/s,
loss=1.1944, acc=0.8367,
auc=0.7348]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Validation: 100%|██████████| 342/342 [01:19<00:00, 4.31it/s,
loss=1.1944, acc=0.8367, auc=0.7348]
Epoch 5/35 [Train]: 0%|██████████| 0/1368 [00:00<?,
?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
multithreaded code, and JAX is multithreaded, so this will likely lead
to a deadlock.
    self.pid = os.fork()
Epoch 5/35 [Train]: 100%|██████████| 1368/1368 [07:13<00:00,
3.37it/s, loss=0.8070, acc=0.8768,
auc=0.8930]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:
RuntimeWarning: os.fork() was called. os.fork() is incompatible with
```

multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 5/35 [Train]: 100%|██████████| 1368/1368 [07:13<00:00, 3.16it/s, loss=0.8070, acc=0.8768, auc=0.8930]

Validation: 0%|██████████| 0/342 [00:00<?,

?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.38it/s, loss=1.3933, acc=0.8054,

auc=0.7212]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.31it/s, loss=1.3933, acc=0.8054, auc=0.7212]

Epoch 6/35 [Train]: 0%|██████████| 0/1368 [00:00<?,

?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 6/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00, 3.38it/s, loss=0.5585, acc=0.9183,

auc=0.9514]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Epoch 6/35 [Train]: 100%|██████████| 1368/1368 [07:12<00:00, 3.16it/s, loss=0.5585, acc=0.9183, auc=0.9514]

Validation: 0%|██████████| 0/342 [00:00<?,

?it/s]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.39it/s, loss=2.0800, acc=0.8048,

auc=0.6762]/opt/conda/lib/python3.10/multiprocessing/popen_fork.py:66:

RuntimeWarning: os.fork() was called. os.fork() is incompatible with multithreaded code, and JAX is multithreaded, so this will likely lead to a deadlock.

```
self.pid = os.fork()
```

Validation: 100%|██████████| 342/342 [01:19<00:00, 4.31it/s,
loss=2.0800, acc=0.8048, auc=0.6762]